Building a Small LLM from Scratch: A Deep Dive into Transformer Foundations, Causal Language Modeling, and Scaled Dot-Product Attention

Authors / Team: Muhammad Umar (Student ID: 23K-0023), Raghib Rizwan Rabbani (Student ID: 23K-0012), Syed Fahad Fahaeem Shah (Student ID: 23K-0062)

Dataset: Tiny Shakespeare (40,000 lines, ~1.11 MB)

Architecture: Decoder-Only Causal Transformer (MiniGPT)

Deliverables: Open Repository, Trained Checkpoints, Interactive Colab, REST API, Unit Test Suite

Abstract

In this investigation, we design, implement, train, evaluate, and package a miniature GPT-style causal language model from scratch in PyTorch without loading any pretrained model weights or relying on black-box abstractions. We examine why the Transformer architecture revolutionized sequence modeling by replacing sequential recurrence with parallel self-attention. We provide explicit manual hand calculations verifying scaled dot-product attention and causal masking, track tensor shapes throughout multi-head projections, train a 0.81M-parameter model on the Tiny Shakespeare corpus, perform four controlled ablation studies (depth, heads, positional encoding, and context length), conduct a rigorous failure analysis, and package the final model behind a production-grade FastAPI service and CLI.

1. Problem and Scenario

Modern large language models (LLMs) have achieved state-of-the-art results across natural language understanding and generation tasks. However, training models with tens or hundreds of billions of parameters requires multi-million dollar compute budgets.

In our applied engineering scenario, an organization maintains a private, specialized domain corpus that is too small to justify training a large foundation model, yet contains distinct stylistic conventions, vocabulary, and syntactic patterns. The engineering objective is to investigate:

> *Can a compact Transformer trained entirely from scratch learn the autoregressive next-token structure of a domain corpus well enough to generate coherent text?*

To answer this question affirmatively and rigorously, we built an end-to-end causal language modeling pipeline:

```
Raw Domain Corpus (Tiny Shakespeare)
  |
  ▼
Deterministic Character Tokenizer (Vocab Size = 65)
  |
  ▼
Contiguous Sequences & Shifted Targets (x_t, y_t = x_{t+1})
  |
  ▼
Token Embeddings + Sinusoidal Positional Encodings
  |
  ▼
Stack of 4 Pre-LayerNorm Causal Transformer Blocks (MHA + GELU FFN)
  |
  ▼
Final Layer Normalization & Language Modeling Head
  |
  ▼
Vocabulary Logits → Cross-Entropy Loss / Autoregressive Generation
```

(A standalone visual schematic of the Pre-LayerNorm Transformer Block and full model dataflow is available at `outputs/figures/architecture_diagram.png`*.)*

---

2. Transformer Foundations

2.1 Sequential vs. Parallel Depth: Why Transformers?

Prior to the Transformer (Vaswani et al., 2017), sequence modeling relied primarily on Recurrent Neural Networks (RNNs), LSTMs, and GRUs. Recurrent models maintain a hidden state (h_t) updated sequentially: [h_t = f(h_{t-1}, x_t)]

This sequential dependency introduces fundamental computational and optimization bottlenecks: 1. **Lack of Parallelization during Training:** Computing (h_t) strictly requires (h_{t-1}). Consequently, training across sequence length (T) incurs (O(T)) sequential depth, preventing full hardware saturation on modern parallel accelerators (GPUs/TPUs/Multi-core CPUs). 2. **Vanishing and Exploding Gradients:** Backpropagation Through Time (BPTT) requires repeated matrix multiplications across all (T) time steps, causing gradients to vanish or explode when modeling long sequences. 3. **Information Bottleneck:** The final hidden state must compress the entire historical context into a fixed-dimensional vector.

The Transformer replaces sequential recurrence with **Scaled Dot-Product Self-Attention**, reducing sequential path length between any two arbitrary tokens from (O(T)) to (O(1)). All tokens in a sequence are processed simultaneously during training, achieving full tensor-level parallel depth.

Metric	Recurrent (RNN / LSTM)	Transformer Self-Attention
Sequential Training Steps	(O(T))	(O(1))
Computational Complexity per Layer	(O(T \cdot d^2))	(O(T^2 \cdot d))
Maximum Interaction Path Length	(O(T))	(O(1))
Hardware Parallelism	Poor (sequential dependency)	High (dense matrix multiplications)

2.2 Scaled Dot-Product Self-Attention

Self-attention maps a query vector and a set of key-value pairs to an output. Given queries (Q \in \mathbb{R}^{T \times d_k}), keys (K \in \mathbb{R}^{T \times d_k}), and values (V \in \mathbb{R}^{T \times d_v}):

[\text{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{Q K^T}{\sqrt{d_k}} + M\right) V]

- **Dot-Product Similarity ((Q K^T)):** Measures pairwise relevance between query token (i) and key token (j).
- **Scaling Factor ((1/\sqrt{d_k})):** For large (d_k), dot products grow large in magnitude, pushing the softmax function into regions with near-zero gradients. Scaling by (\sqrt{d_k}) preserves unit variance and ensures stable gradients.
- **Causal Mask ((M)):** For autoregressive language modeling, (M_{i,j} = 0) if (j \le i), and (M_{i,j} = -\infty) if (j > i). This prevents tokens at step (i) from attending to future tokens (j > i).
- **Softmax:** Normalizes attention scores into a valid probability distribution across keys for each query row ((\sum_j W_{i,j} = 1.0)).
- **Weighted Value Sum ((W V)):** Computes contextual token representations as convex combinations of value vectors.

2.3 Required Hand Calculations (Handout Section 6)

To demonstrate exact mathematical understanding, we execute a manual hand calculation for a 3-token sequence with (d_k = 2) and (d_v = 2).

Step 1: Explicit Input Matrices

Let the Query ((Q)), Key ((K)), and Value ((V)) matrices for 3 tokens be: [Q =

$$\begin{bmatrix}1.0 & 0.0\\ 0.0 & 2.0\\ 1.0 & 1.0\end{bmatrix}$$

, \quad K =

$$\begin{bmatrix}1.0 & 1.0\\ 0.0 & 1.0\\ 2.0 & 0.0\end{bmatrix}$$

, \quad V =

$$\begin{bmatrix}1.0 & 2.0\\ 3.0 & 0.0\\ 0.0 & 4.0\end{bmatrix}$$

]

Step 2: Compute Dot-Product Scores ((Q K^T))

[K^T =

$$\begin{bmatrix}1.0 & 0.0 & 2.0 & 1.0 & 1.0 & 0.0\end{bmatrix}$$

] [Q K^T =

$$\begin{bmatrix}1.0 & 0.0 & 0.0 & 2.0 & 1.0 & 1.0\end{bmatrix}$$

$$\begin{bmatrix}1.0 & 0.0 & 2.0 & 1.0 & 1.0 & 0.0\end{bmatrix}$$

=

$$\begin{bmatrix}1.0 & 0.0 & 2.0 & 2.0 & 2.0 & 0.0 & 2.0 & 1.0 & 2.0\end{bmatrix}$$

]

Step 3: Divide by (\sqrt{d_k} = \sqrt{2} \approx 1.4142)

[S = \frac{Q K^T}{\sqrt{2}} =

$$\begin{bmatrix}0.7071 & 0.0000 & 1.4142 & 1.4142 & 1.4142 & 0.0000 & 1.4142 & 0.7($$

]

Step 4: Apply Row-Wise Softmax (Unmasked)

[\mathrm{Softmax}(S)*{i,j} = |frac{e^{S*] - }}}{\sum_k e^{S_{i,k}}**Row 1:** (e^{0.7071} \approx 2.028), (e^{0.0} = 1.0), (e^{1.4142} \approx 4.113). (\sum = 7.141). (W_{\text{unmasked}}[0] = [0.284, 0.140, 0.576]) - **Row 2:** (e^{1.4142} \approx 4.113), (e^{1.4142} \approx 4.113), (e^{0.0} = 1.0). (\sum = 9.227). (W_{\text{unmasked}}[1] = [0.446, 0.446, 0.108]) - **Row 3:** (e^{1.4142} \approx 4.113), (e^{0.7071} \approx 2.028), (e^{1.4142} \approx 4.113). (\sum = 10.255). (W_{\text{unmasked}}[2] = [0.401, 0.198, 0.401])

[W_{\text{unmasked}} =

$$\begin{bmatrix}0.284 & 0.140 & 0.576 & 0.446 & 0.446 & 0.108 & 0.401 & 0.198 & 0.40$$

]

Step 5: Unmasked Weighted Output ((W_{\text{unmasked}} V))

[\text{Out}*{|text{unmasked}} = W]}}

$$\begin{bmatrix}1.0 & 2.0 & 3.0 & 0.0 & 0.0 & 4.0\end{bmatrix}$$

= \begin{bmatrix} 0.704 & 2.872 \ 1.783 & 1.326 \ 0.994 & 2.406 \end{bmatrix

Step 6: Apply Causal Masking

In causal autoregressive modeling, future tokens cannot be attended to. Positions (j > i) are set to (-\infty): [S_{\text{causal}} =

$$\begin{bmatrix} 0.7071 & -\infty & -\infty & 1.4142 & 1.4142 & -\infty & 1.4142 & 0.7071 & 1.$$

]

Applying row-wise softmax: - **Row 1:** Only token 1 is visible (\to) weight is strictly ([1.000, 0.000, 0.000]). - **Row 2:** Tokens 1 and 2 are equal ((1.4142)) (\to) ([0.500, 0.500, 0.000]). - **Row 3:** Full historical context visible (same as unmasked Row 3) (\to) ([0.401, 0.198, 0.401]).

[W_{\text{causal}} =

$$\begin{bmatrix} 1.000 & 0.000 & 0.000 & 0.500 & 0.500 & 0.000 & 0.401 & 0.198 & 0.40$$

]

Multiplying by (V): [\text{Out}*{|text{causal}} = W*]}} V =

$$\begin{bmatrix} 1.000 & 0.000 & 0.000 & 0.500 & 0.500 & 0.000 & 0.401 & 0.198 & 0.40 \end{bmatrix}$$

$$\begin{bmatrix} 1.0 & 2.0 & 3.0 & 0.0 & 0.0 & 4.0 \end{bmatrix}$$

= \begin{bmatrix} 1.000 & 2.000 \ 2.000 & 1.000 \ 0.994 & 2.406 \end{bmatrix

Step 7: Information Flow Analysis

- **Token 1:** In the unmasked setting, Token 1 received 57.6% of its information from future Token 3. Under the causal mask, Token 1 receives 100% of its representation from its own value vector, preserving strict temporal isolation.
- **Token 2:** In the unmasked setting, Token 2 received 10.8% of its information from future Token 3. Under the causal mask, Token 2 aggregates information only from Token 1 and Token 2 (50% each).
- **Token 3:** Because Token 3 is at the final position, its receptive field spans the complete history in both masked and unmasked variants.

2.4 BERT Masked LM (Bidirectional) vs. GPT Causal LM (Autoregressive)

A key conceptual comparison required by Section 3 & 5.3 is the distinction between BERT and GPT objectives: 1. **BERT (Masked Language Modeling):** - **Attention:** Fully bidirectional (all tokens attend to all other tokens, past and future). - **Training Objective:** 15% of tokens are corrupted (replaced with `[MASK]`, random token, or kept unchanged), and the model predicts only the masked tokens given full bidirectional context. - **Use Case:** NLU tasks (classification, named entity recognition, question answering). Unsuitable for open-ended autoregressive text generation because generation would require unmasking tokens iteratively with no clear causal direction. 2. **GPT (Causal Language Modeling):** - **Attention:** Strictly causal (lower-triangular mask). - **Training Objective:** Next-token prediction across all (T) positions: (\mathcal{L} = -\sum_{t=1}^T \log P(x_t \mid x_{<t})). - **Use Case:** Natural text generation, conversational agents, code generation.

---

3. Architecture and Implementation Details

3.1 Tokenizer Design

We implement `CharacterTokenizer` (`src/tokenizer.py`), mapping the 65 unique ASCII/Unicode characters in Tiny Shakespeare to unique integer IDs ([0, 64]) with a deterministic `<unk>` token: - **Determinism:** Vocabulary built by lexicographically sorting unique characters. - **Reversibility:** Bijective mappings `encode(s)` and `decode(ids)`. - **Serializability:** Stored as JSON (`checkpoints/tokenizer.json`) for seamless reload.

3.2 Positional Encodings: Sinusoidal & Learned

Self-attention is permutation-equivariant. To encode word order, we implement: [PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i / d_{model}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i / d_{model}}}\right)]

3.3 Multi-Head Attention (MHA)

`MultiHeadAttention` (`src/attention.py`) splits queries, keys, and values into (h=4) parallel attention heads ((d_k = d_v = 128 / 4 = 32)): 1. **Linear Projections:** (Q = X W_q, K = X W_k, V = X W_v \in \mathbb{R}^{B \times T \times d_{model}}). 2. **Head**

Splitting: Reshaped to ((B, T, h, d_k)) and transposed to ((B, h, T, d_k)). 3. **Scaled Dot-Product per Head:** Applied in parallel with registered lower-triangular causal mask buffer. 4. **Merge & Project:** Output transposed to ((B, T, h, d_k)), flattened to ((B, T, d_{model})), and projected via (W_o).

3.4 Pre-LayerNorm Transformer Block

Modern architectures (GPT-2, LLaMA) place Layer Normalization **before** each sublayer rather than after (post-LN): [x^{(1)} = x + \text{MHA}(\mathrm{LN}_1(x))] [x^{(2)} = x^{(1)} + \text{FFN}(\mathrm{LN}_2(x^{(1)}))] [\text{FFN}(z) = \text{Linear}_2(\mathrm{GELU}(\text{Linear}_1(z)))]

Pre-LN provides an unimpeded identity residual pathway from input to output, preventing vanishing gradients and allowing smooth training without delicate learning rate warmups.

---

4. Dataset and Preprocessing

The **Tiny Shakespeare** dataset contains 1,115,394 characters (~40,000 lines) across Shakespearean plays: - **Split:** 90% Training split (1,003,854 characters) and 10% Validation split (111,540 characters). - **Batching:** Mini-batches of (B=64) sequences of fixed context length (T=128). - **Target Shifting:** For input (X = [x_0, x_1, \dots, x_{T-1}]), target (Y = [x_1, x_2, \dots, x_T]).

---

5. Training Setup and Hyperparameters

- **Hardware:** Multi-core CPU platform (28 logical cores) utilizing PyTorch intra-op multi-threading.
- **Optimizer:** AdamW ((\beta_1=0.9, \beta_2=0.99, \text{weight_decay}=0.01)).
- **Learning Rate Schedule:** Linear warmup over 100 steps to peak (1\times 10^{-3}), followed by cosine decay down to (1\times 10^{-4}).
- **Gradient Clipping:** Maximum grad norm clipped at (1.0).
- **Total Iterations:** 1,500 iterations ((B=64, T=128), processing 8,192 tokens per step).

---

6. Experimental Results

6.1 Training & Validation Convergence

- **Initial Loss:** Train 4.2006, Val 4.2026 (matching theoretical random baseline (-\ln(1/65) \approx 4.17)).
- **Final Loss:** Train 1.8340, Val 1.8812.
- **Perplexity:** Reduced from (66.8) to (\mathbf{6.56}).
- **Total Wall-Clock Time:** ~5.4 minutes.
- **Model Parameters & Layer Breakdown:** 805,632 parameters *(see Figure 5 (`outputs/figures/figure5_model_summary.png`) for the captured model summary and layer breakdown card).*

6.2 Generation Quality Across 5 Fixed Prompts (Figure 8)

1. **Prompt 1 (`KING LEAR:\n`):**
   Generated:
   `text KING LEAR: I see the my from grown, How it the or theinge; what ay, our me so boria, I do yet he my she go her`
2. **Prompt 2 (`ROMEO:\n`):**
   Generated:
   ```text ROMEO: O, sir, not take and been be him: to-f us this barwile.

HENRY VI: And a meak of scome is putt not h `3. **Prompt 3 (`To be, or not to be, that is the `):** *Generated:*` text To be, or not to be, that is the shall, And I shall she drest death me of unto with his Take a fatught the havinester, but out will y `4. **Prompt 4 (`First Citizen:\nBefore we proceed any further, `):** *Generated:*` text First Citizen: Before we proceed any further, then heaven you spike of have any agring of Moysence: The part and heaved see be roth seeched all of `5. **Prompt 5 (`The future of artificial intelligence `):** *Generated:*` text The future of artificial intelligence forther, And all thee lass of thee been to Rereit your blorford.

HENRY BOKE: You make would Some I ```
   ```

7. Controlled Ablation Studies (Figure 9)

We executed four controlled ablation experiments (visualized together in Figure 9, with dedicated sequence scaling telemetry in `outputs/figures/context_length_ablation.png`):

Ablation 1: Depth (Layers (N \in {2, 4, 6}))

Layers ((N))	Parameters	Validation Loss	Training Time (200 steps)
2 Layers	411,392	2.6985	51.5s
4 Layers (Baseline)	805,632	2.6510	124.4s
6 Layers	1,199,872	2.7160	160.6s
Interpretation: 4 layers achieved the best validation loss. Surprisingly, 6 layers overfits slightly at only 200 steps — the increased capacity demands more iterations to converge — confirming that depth alone does not guarantee improvement under a fixed short training budget.			

Ablation 2: Multi-Head Splitting ((h \in {2, 4, 8}))

Heads ((h))	Head Dim ((d_k))	Parameters	Validation Loss
2 Heads	64	805,632	2.6622
4 Heads (Baseline)	32	805,632	2.6510
8 Heads	16	805,632	2.6591
Interpretation: 4 heads achieved the optimal trade-off. 8 heads over-partitions the 128-dimensional embedding ((d_k=16)), degrading per-head representational capacity. All three configurations share identical parameter counts as only the head-splitting structure changes.			

Ablation 3: Positional Encoding (Sinusoidal vs. Learned)

Positional Strategy	Parameters	Validation Loss
Sinusoidal (Vaswani et al.)	805,632	2.6510
Learned (GPT-2 style)	822,016	2.4203
Interpretation: Learned position embeddings achieved substantially lower loss (−0.231), adding only 16,384 parameters (one entry per position × 128 dimensions). This gap suggests positional structure is actively learned rather than merely injected at the 200-step mark, though sinusoidal encodings remain preferable when sequence-length generalization is required.		

Ablation 4: Context Length ((T \in {64, 128, 256}) Tokens, Batch Size = 64)

Context Length ((T))	Parameters	Validation Loss	Throughput (tok/s)	Training Time (200 steps)	Peak RSS (MB)
64 Context	805,632	2.6430	1,418.3	577.6s (~9.6m)	315.9 MB
128 Context (Baseline)	805,632	2.6510	1,282.6	1,277.4s (~21.3m)	333.3 MB
256 Context	805,632	2.6714	1,078.4	3,038.6s (~50.6m)	219.1 MB
Interpretation: All three runs were executed strictly with an identical batch size of 64 across 200 iterations for methodological consistency. As context length quadruples from 64 to 256 tokens, training time grows super-linearly from 577.6s to 3,038.6s due to the quadratic (O(T^2)) computational complexity of scaled dot-product attention on CPU, resulting in a gradual decrease in throughput from 1,418.3 to 1,078.4 tokens/second. The validation					

Context Length ((T))	Parameters	Validation Loss	Throughput (tok/s)	Training Time (200 steps)	Peak RSS (MB)
losses remain tightly clustered around 2.64–2.67 across 200 steps.					

---

8. Failure Mode Analysis

1. **Failure Mode 1: Degenerate Repetition under Greedy Decoding ((T=0.0)):**
   Pure argmax selection leads the model to fall into cyclical attractor states (e.g., repeating `"and so and so and so"`), as small local loops become reinforced. *Mitigation:* Top-K ((K=40)) and temperature sampling ((T=0.8)) break greedy attractors.
2. **Failure Mode 2: Semantic Hallucination on Modern / Out-of-Domain Prompts:**
   Prompting with modern technical concepts like `"artificial intelligence"` causes the model to hallucinate Shakespearean phrasing (`"art"` , `"thee"` , `"fair lord"`), reflecting the severe domain limitation of the training corpus.

---

9. Engineering Packaging: CLI & FastAPI Deployment

The model is packaged into two production-grade interfaces: 1. **Command-Line Interface (CLI):** `bash python -m src.generate --prompt "The future of artificial intelligence" --max-new-tokens 80 --temperature 0.8` *(See Figure 10 (`outputs/figures/figure10_cli_inference.png`) for the captured CLI inference terminal verification card).* 2. **FastAPI REST Service:** `bash uvicorn src.api:app --host 0.0.0.0 --port 8000` Exposes endpoints: - `GET /health` : Health and device

telemetry. - `GET /info`: Architecture and parameter breakdown. - `POST /generate`: Fast autoregressive inference with JSON input and output.

---

10. Conclusion

This project demonstrated the end-to-end viability of training a compact 0.81M-parameter Transformer causal language model from scratch. By validating scaled dot-product attention through hand calculations, verifying tensor shapes, enforcing causal masking, and analyzing ablations, we demonstrated why modern LLMs are capable of learning rich domain syntax and autoregressive generation without reliance on external pretrained weights.

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS 2017), 30. arXiv:1706.03762.
2. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Technical Report (GPT-2).
3. Karpathy, A. (2022). *nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs*. GitHub repository: https://github.com/karpathy/nanoGPT.
4. Karpathy, A. (2023). *Let's build GPT: from scratch, in code, spelled out*. YouTube Tutorial & Video Lecture: https://youtu.be/kCc8FmEb1nY.
5. Hendrycks, D., & Gimpel, K. (2016). *Gaussian Error Linear Units (GELUs)*. arXiv preprint arXiv:1606.08415.
6. Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer Normalization*. arXiv preprint arXiv:1607.06450.
7. Loshchilov, I., & Hutter, F. (2017). *Decoupled Weight Decay Regularization (AdamW)*. International Conference on Learning Representations (ICLR 2019). arXiv:1711.05101.
8. Umar, M., Rabbani, R. R., & Shah, S. F. F. (2026). *Building a Small LLM from Scratch: A Deep Dive into Transformer*

Foundations, Causal Language Modeling, and Scaled Dot-Product Attention. Medium. https://medium.com/@muhammad.umar26804/building-a-small-llm-from-scratch-a-deep-dive-into-transformer-foundations-causal-language-7b7fb702a878.